Báo cáo nhóm bài tập lớn

Báo cáo chung của nhóm cho đề 06, nộp kèm 4 báo cáo cá nhân và hồ sơ minh chứng. Hệ thống minh hoạ là Hanni, web học tiếng Trung đang chạy thật. Mọi số liệu trong báo cáo lấy từ tệp đo gốc trong thư mục data/raw/ của hồ sơ, không có số nào gõ tay.

Thông tin nhóm và đề tài

Lớp: ..... Nhóm: .....

Mã và tên đề tài: Đề 06 – Thiết kế nền tảng học trực tuyến cho 1.000 sinh viên

Thời gian thực hiện: từ 05/11/2026 đến 23/11/2026

Họ và tên	Mã sinh viên	Vị trí trong phiếu đề	Phần việc chính
Phạm Hải Đăng		Sinh viên 1: kiến trúc, bảng tính tài nguyên	Bảng tính tài nguyên, kiến trúc, lab, API học liệu, giám sát, cải tiến theo điểm nghẽn
Lê Đình Thành		Sinh viên 2: triển khai Web học liệu	3 học liệu, trang /hoc-lieu, phát học liệu qua Nginx (vai CDN), đo phía trình duyệt
Bùi Thị Vân Anh		Sinh viên 3: tạo tải, thu thập số liệu	Kịch bản tải k6, Kiểm thử 1 và 2, tổng hợp số liệu
Chu Hồng Đức		Sinh viên 4: sao lưu, phục hồi, đánh giá mở rộng	Sao lưu, Kiểm thử 3, kiểm thử phân quyền, đánh giá mở rộng, README

Tóm tắt kết quả

- 1.000 tài khoản không phải 1.000 người cùng lúc: tự học rải rác khoảng **63 người cùng lúc**, cả lớp xem video 300, thi thử đồng loạt 600.
- Phần tốn nhất là **băng thông video** (khoảng 888 Mbps khi cả lớp xem), nên video không đi qua máy chủ API mà do Nginx/CDN phát.
- Kiểm thử 2 trên lab: 3 mức tải khoảng 50, 120, 180 request/giây đều đạt (95% lượt đọc dưới 300 ms).
- Điểm nghẽn đầu tiên là hàng chờ kết nối CSDL. Sau khi sửa, mức tối đa còn nhanh tăng từ 157 lên 190 request/giây; lúc quá tải, thời gian chờ giảm từ 5,0 giây còn 0,73 giây.
- Kiểm thử 3: dừng hẳn API thì cảnh báo kêu sau khoảng 1 phút, bật lại có trả lời sau 4–5 giây; mất dữ liệu thì khôi phục xong trong 52 giây, học liệu đúng 30/30 tệp.

Hồ sơ nộp kèm

Thư mục hoặc tệp	Nội dung (theo mục "Hồ sơ nộp chung" của phiếu đề)
bao-cao/	Báo cáo nhóm (tệp này) và 4 báo cáo cá nhân theo mẫu
so-do-va-bang/	Sơ đồ hệ thống, bảng tài nguyên, IP và phân quyền, bảng tính tài nguyên
cau-hinh-ma-nguon/	Cấu hình, script và mã nguồn nhóm viết cho đề này
README.md	Phiên bản, cách dựng lại hệ thống, cách chạy kiểm thử

Thư mục hoặc tệp	Nội dung (theo mục "Hồ sơ nộp chung" của phiếu đề)
bang-kiem-thu.xlsx	Bảng kiểm thử: đầu vào, thao tác, mong đợi, thực tế, đạt/chưa đạt, minh chứng có giờ chạy
data/	Dữ liệu đo gốc (không sửa tay) và bảng tổng hợp
evidence/	Minh chứng theo ca kiểm thử: nhật ký, ảnh chụp, danh sách mã băm
video-demo.mp4	Video demo 5–7 phút

Yêu cầu, giả định và tính toán

1 Yêu cầu và phạm vi

1.1 Bài toán

Đề 06 yêu cầu thiết kế nền tảng học trực tuyến cho 1.000 tài khoản sinh viên. Trọng tâm là thiết kế hệ thống: tính tải, chọn kiến trúc, đo kiểm và khôi phục khi hỏng. Hanni đã có sẵn bài học, ôn tập, thi thử, ghi âm phát âm, nên nhóm không viết lại ứng dụng mà chứng minh bằng số: hệ thống cần gì cho 1.000 tài khoản, điểm nghẽn ở đâu, và khôi phục được không.

Yêu cầu của đề	Nhóm đáp ứng bằng	Mục
Yêu cầu 1: thiết kế web, dữ liệu, video, đăng nhập; phân biệt tổng tài khoản với số người dùng cùng lúc	Kiến trúc 3 tầng; tính số người cùng lúc cho 3 tình huống	2, 3
Yêu cầu 2: tính băng thông, lưu trữ theo ít nhất 2 tình huống; công khai bitrate, thời lượng, tỷ lệ cùng lúc, dự phòng	3 tình huống, bảng giả định công khai	2
Yêu cầu 3: Web học liệu có ít nhất 3 tài nguyên; thử 3 mức tải phù hợp thiết bị	Trang /hoc-lieu với video, PDF, MP3; lab k3s	4, 5
Yêu cầu 4: đo thời gian trả lời, thông lượng, lỗi, CPU/RAM; mở rộng theo điểm nghẽn; sao lưu, kiểm soát truy cập	k6, Prometheus, Grafana; sao lưu, phân quyền	5, 6
Kiểm thử 1: máy khách tải đúng học liệu	Tải đủ tệp, so mã băm SHA-256	5.2
Kiểm thử 2: 3 mức tải, điều kiện đo lặp lại được	Script chạy tự động, mỗi mức 3 lần	5.3
Kiểm thử 3: dừng dịch vụ rồi khôi phục, kiểm tra học liệu và thời gian gián đoạn	Dừng API, mất dữ liệu, tắt từng thành phần	5.4

1.2 Phạm vi và quy tắc

- Trong phạm vi: thiết kế cho 1.000 tài khoản, môi trường đo riêng (lab), trang học liệu, số đo gốc, diễn tập khôi phục, đề xuất mở rộng.
- Ngoài phạm vi: đo trên web thật đang có người dùng, dữ liệu thật, thuê Cloud trả phí.
- Chỉ dùng dữ liệu giả: 1.000 tài khoản loadtest-*@example.test. Chỗ nào chưa đo thì ghi rõ là chưa đo.

2 Giả định và tính tài nguyên

2.1 Tổng tài khoản khác số người dùng cùng lúc

1.000 tài khoản không có nghĩa 1.000 người dùng cùng lúc. Khi sinh viên tự học rải rác, số người cùng lúc = số lượt vào học mỗi phút × số phút mỗi phiên (định luật Little). Giả định 300 người học mỗi ngày, một nửa rơi vào 1 giờ cao điểm, nên có 2,5 lượt vào mỗi phút; mỗi phiên 25 phút → khoảng **63 người cùng lúc**. Hai tình huống còn lại do giảng viên tạo ra (cả lớp cùng xem, cùng thi) nên tính thẳng theo tỷ lệ.

Nhóm	Giả định	Giá trị	Căn cứ
Video	Bitrate 540p / 720p	2,0 / 3,0 Mbps	Hướng dẫn HLS của Apple
Video	Tiếng kèm video	0,128 Mbps	AAC 128 kbit/s
Video	Tỷ lệ xem 540p / 720p	60% / 40%	Giả định
Video	Trung bình 1 người xem	2,528 Mbps	Tính từ 3 dòng trên
Phiên học	Request tới API	0,25 request/giây, 8 KB	Giả định dư, đối chiếu số đo ở mục 2.4
Phiên học	Tệp ghi âm	0,125 request/giây, 4 KB	Dung lượng đo trên 10.900 tệp
Chung	Dự phòng	30%	Cho lúc đông đột biến và sai số

2.2 Ba tình huống sử dụng

Chỉ tiêu	Tự học	Cả lớp xem video	Thi thử đồng loạt
Thời lượng	Phiên 25 phút, rải rác trong ngày	Buổi 45 phút, 8 buổi mỗi tháng	Mở đề trong 60 giây, nộp bài trong 30 giây
Tỷ lệ cùng lúc	6,3%	30%	60%
Người dùng cùng lúc	**63**	**300**	**600**
Người đang xem video	18,9	270	0
Băng thông, đã cộng dự phòng	**63,3 Mbps**	**888,1 Mbps**	**6,2 Mbps**
Request tới API	11,0/giây	7,5/giây	10,0/giây lúc mở đề, 20,0/giây lúc nộp
Lưu lượng video mỗi tháng	1.280 GB	1.843 GB	0

Thi thử tốn ít băng thông vì lúc làm bài, đáp án chỉ giữ ở trình duyệt; máy chủ chỉ chịu 2 đợt: mở đề và nộp bài.

2.3 Lưu trữ

Hạng mục (GB)	Không tự lưu video	Khoá nhỏ 40 video	Đủ lộ trình 120 video
Video HLS 2 mức, 15 phút mỗi video	0	23,7	71,0
Video bản gốc 8 Mbps	0	36,0	108,0
PDF, ảnh đại diện, ghi âm	0,5	0,5	0,5
CSDL sau 1 năm	2,0	2,0	2,0
14 bản sao lưu CSDL	8,3	8,3	8,3
Tổng, kèm 1 bản sao nội dung và dự phòng 30%	**14,8**	**169,9**	**480,1**

2.4 Đối chiếu giả định với số đo thật

Ngày 07/11 nhóm ghi lại mọi request của một phiên học trọn bài 1 trên trình duyệt (tệp HAR, data/raw/har/phien-hoc.har, đã xoá cookie): **20 request tới API, tổng 61 KB, trung bình 2,0 KB mỗi request**, cộng 10 tệp ghi âm 53 KB. Số thật nhỏ hơn giả định 8 KB bốn lần, nên băng thông API trong bảng tính là con số dư, an toàn. Tần suất request tuỳ người học nhanh hay chậm nên nhóm giữ 0,25 request/giây.

2.5 Kết luận từ bảng tính

- Ở 1.000 tài khoản, phần khó là **băng thông video**, không phải API: cả lớp xem video cần khoảng 888 Mbps trong khi API chỉ khoảng 56 request/giây.
- Nếu không tự lưu video thì dung lượng rất nhỏ, khoảng 14,8 GB.
- Rủi ro thật là các đợt đồng loạt (cả lớp mở đề, nộp bài, đăng nhập), không phải số trung bình.

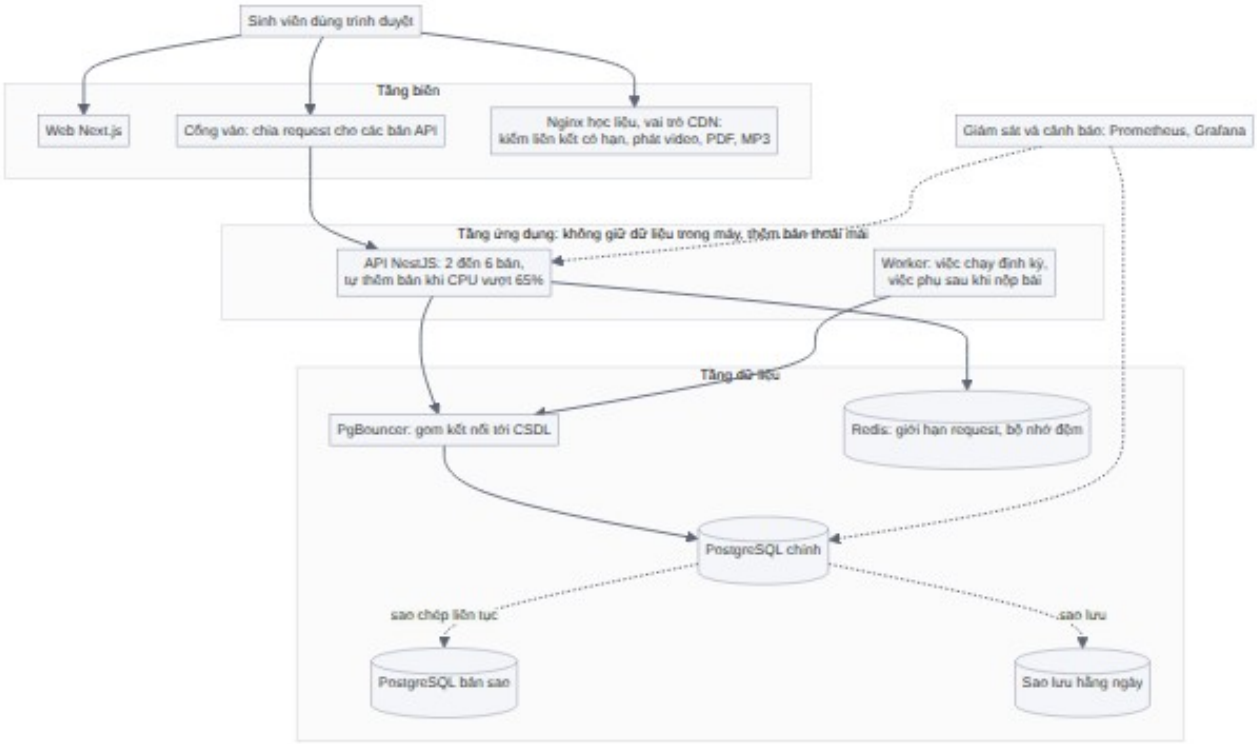
Kiến trúc và triển khai

3 Kiến trúc

3.1 Hiện trạng

Hanni đang chạy trên 1 máy chủ ảo 2 nhân, 3,8 GB RAM: 1 tiến trình NestJS, PostgreSQL 16 và Redis 7 trong Docker, web Next.js trên Vercel, video nhúng YouTube. Rà mã nguồn thấy các chỗ sẽ hỏng khi đông người: giới hạn request theo địa chỉ IP nên cả phòng máy dùng chung 1 IP bị chặn; bộ đếm nằm trong bộ nhớ của 1 tiến trình nên không chạy được nhiều bản; việc định kỳ chạy chung với API; kiểm tra sức khoẻ luôn báo tốt kể cả khi CSDL đã chết.

3.2 Kiến trúc đề xuất cho 1.000 tài khoản



Hình 1. Kiến trúc 3 tầng: tầng biên, tầng ứng dụng không giữ dữ liệu, tầng dữ liệu (tệp gốc: so-do-va-bang/kien-truc-dich.png)

- **Học liệu không đi qua API:** API chỉ kiểm quyền rồi trả liên kết có hạn 10 phút; tệp do Nginx phát. Khi lên quy mô lớn chỉ cần thay Nginx bằng CDN, không phải sửa API hay trang web.
- **Đăng nhập:** mã đăng nhập (JWT) nằm trong cookie, tự gia hạn. Giới hạn số request đếm theo tài khoản (lưu trong Redis) thay vì theo IP, nên cả phòng máy chung 1 IP không bị chặn.
- **Tầng ứng dụng không giữ dữ liệu trong máy:** mọi dữ liệu ở CSDL hoặc Redis, nên thêm bản API là chạy được ngay; Kubernetes tự thêm bản khi CPU cao.
- **Khi quá tải:** API trả lời ngay "thử lại sau" (mã 503) cho một phần lượt đọc, thay vì để tất cả cùng chờ rồi cùng hỏng.

3.3 Dữ liệu đặt ở đâu

Loại dữ liệu	Ví dụ	Nơi lưu	Sao lưu
Tài khoản, phiên đăng nhập	User, RefreshToken	PostgreSQL	Hằng ngày + nhật ký thay đổi
Tiến độ học	UserWordProgress, UserLessonProgress	PostgreSQL chính	Hằng ngày + nhật ký thay đổi
Nhật ký học (lớn nhanh nhất)	ReviewLog	PostgreSQL, chia theo tháng	Hằng ngày + nhật ký thay đổi
Bài thi thử	MockExamAttempt (mỗi lượt nộp có khoá riêng, gửi lại không thành 2 bài)	PostgreSQL	Hằng ngày + nhật ký thay đổi

Loại dữ liệu	Ví dụ	Nơi lưu	Sao lưu
Nội dung học	Word, Lesson	PostgreSQL, lưu tạm trong API	Có trong mã nguồn, dựng lại được
Bảng xếp hạng	Tính từ tiến độ	Đọc từ bản sao	Dựng lại được
Bộ đếm giới hạn request	Khoá Redis có hạn	Redis	Không cần, tự dựng lại
Học liệu	Video HLS, PDF, MP3 + danh sách mã băm	Ổ tệp sau Nginx (thực tế: kho tệp + CDN)	Nén + mã băm

3.4 Các quyết định chính

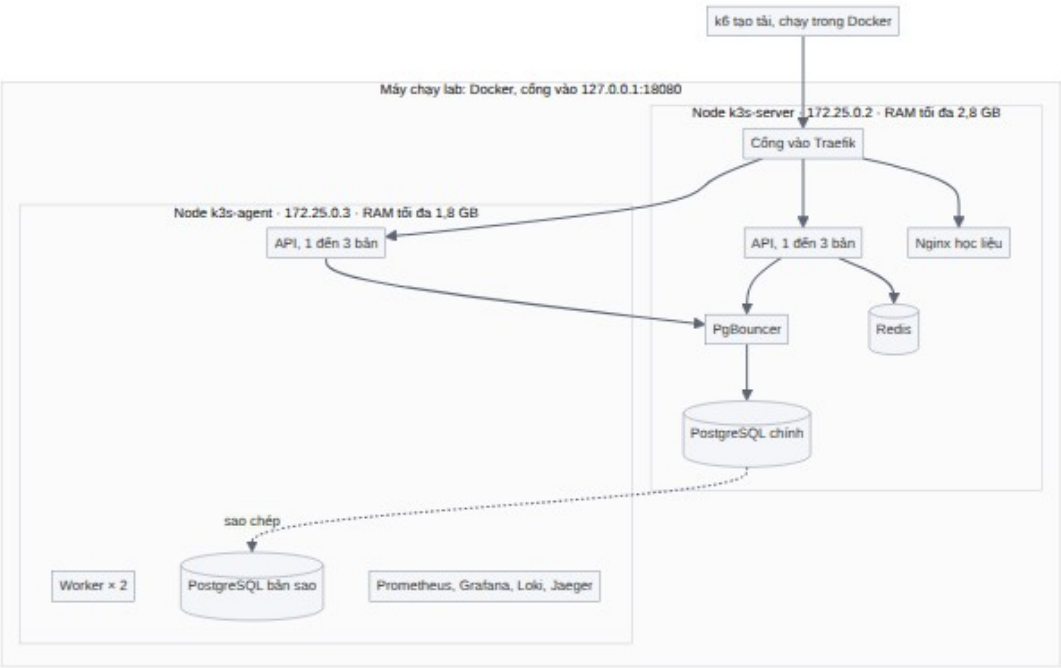
Quyết định	Vì sao	Đánh đổi
Video không đi qua API	Cả lớp xem cần gần 900 Mbps	Phụ thuộc CDN
Giữ 1 ứng dụng (không tách nhiều dịch vụ nhỏ)	Tải ở mức này chưa cần tách	Không mở rộng riêng từng phần
PostgreSQL là nơi giữ dữ liệu thật, Redis chỉ giữ thứ dựng lại được	Mất Redis không mất dữ liệu học	Phải xử lý dữ liệu đệm cũ
Tách worker khỏi API	Chạy nhiều bản API mà việc định kỳ không chạy trùng	Thêm 1 tiến trình phải theo dõi
Giới hạn request theo tài khoản	Không chặn cả phòng máy chung IP	Thêm 1 lượt gọi Redis mỗi request
Đo trong lab k3s có giới hạn CPU/RAM cố định	Lặp lại được, không ảnh hưởng người dùng thật	Không giống hệt môi trường thật
Có bản sao CSDL, nhưng mặc định vẫn đọc bản chính	Không đọc phải dữ liệu cũ ngay sau khi ghi	Bản sao chỉ dùng cho trang xếp hạng
Kiểm tra sức khoẻ chờ đủ lâu trước khi khởi động lại	Tránh giết nhầm bản đang bận lúc quá tải	Bản treo thật phải chờ 1 phút mới khởi động lại

Mọi thay đổi trong mã đều bật bằng biến môi trường, mặc định giữ hành vi cũ, nên web thật không đổi khi gộp mã.

4 Triển khai

4.1 Môi trường lab

Cụm Kubernetes bản nhẹ k3s gồm 2 node, chạy trong Docker. Máy chỉ cần Docker: công cụ quản lý cụm, k6, ffmpeg đều chạy trong container. Dựng lại từ đầu theo README bằng 5 lệnh make; trên máy ảo GitHub mất 322 giây.



Hình 2. Sơ đồ lab: 2 node, địa chỉ IP và các thành phần (tệp gốc: so-do-va-bang/so-do-lab.png)

Thành phần	Địa chỉ hoặc tên miền	Ghi chú
Node k3s-server	172.25.0.2	Điều khiển cụm và chạy ứng dụng, RAM tối đa 2,8 GB
Node k3s-agent	172.25.0.3	Chạy ứng dụng, RAM tối đa 1,8 GB
Mạng nội bộ của pod / của dịch vụ	10.42.0.0/16 / 10.43.0.0/16	Mặc định của k3s
Cổng vào (ingress)	127.0.0.1:18080	Chỉ mở trên máy chạy lab
API	api.hanni.localhost	NestJS, cổng trong cụm 8000
Học liệu	media.hanni.localhost	Nginx kiểm liên kết có hạn
Giám sát	grafana / prometheus / alertmanager / jaeger .hanni.localhost	
Web	hanni.localhost:3000	Cùng tên miền với API để trình duyệt nhận cookie đăng nhập

4.2 Tài nguyên từng thành phần

Thành phần	Số bản	CPU yêu cầu / trần (lõi)	RAM yêu cầu / trần	Cổng
API NestJS	2–6, tự thêm khi CPU vượt 65%	0,25 / 1	256 / 512 MB	8000

Thành phần	Số bản	CPU yêu cầu / trần (lõi)	RAM yêu cầu / trần	Cổng
Worker (việc định kỳ, việc phụ)	2	0,05 / 0,5	180 / 384 MB	8000
PostgreSQL chính	1	0,25 / 1	256 / 512 MB	5432
PostgreSQL bản sao	1	0,1 / 1	128 / 384 MB	5432
PgBouncer ghi / đọc	1 / 1	0,05 / 0,5	16 / 64 MB	5432
Redis	1	0,05 / 0,5	32 / 128 MB	6379
Nginx học liệu	1	0,05 / 1	32 / 128 MB	8080
Sao lưu (chạy 02:00 mỗi ngày)	0 hoặc 1	0,05 / 1	64 / 256 MB	
Prometheus	1	0,05 / 0,5	200 / 400 MB	9090
Grafana, Loki, Jaeger, Alertmanager và công cụ phụ	1 mỗi loại	0,005–0,025 / 0,1–0,5	16–96 / 48–256 MB	

Ổ đĩa: CSDL 2 GB, nhật ký thay đổi (WAL) 2 GB, bản sao 2 GB, sao lưu 2 GB. PgBouncer gom tối đa 70 kết nối của các bản API về 20 kết nối thật tới PostgreSQL.

4.3 Phân quyền

Tài nguyên	Khách	Người học	Giáo viên	Quản trị
Danh sách học liệu	Xem	Xem	Xem	Xem
Video bài giảng (R1)	Được mời đăng nhập	Xem	Xem	Xem
PDF (R2), bài nghe MP3 (R3)	Tải	Tải	Tải	Tải
Tiến độ học	Từ chối	Chỉ của mình	Học viên lớp mình	Theo nhu cầu
Trang số liệu nội bộ /api/metrics	Không thấy (404)	Không thấy (404)	Không thấy (404)	Chỉ trong cụm
CSDL, Redis	Không tới được	Không tới được	Không tới được	Chỉ trong cụm

Trong cụm, mặc định chặn mọi kết nối vào rồi chỉ mở đúng đường cần: cổng vào tới API và Nginx; API, worker tới Redis và PgBouncer; chỉ PgBouncer tới PostgreSQL; chỉ phần giám sát được lấy số liệu.

4.4 Web học liệu

Trang /hoc-lieu trong hanni-client; lối vào là 1 ô ở trang Luyện tập, không thêm mục menu. 3 tài nguyên nhóm tự làm, không dính bản quyền:

Mã	Tài nguyên	Định dạng	Dung lượng	Quyền
R1	Bài giảng "Bốn thanh điệu tiếng Trung", 66 giây	Video HLS 540p 2 Mbps + 720p 3 Mbps, đoạn 6 giây; thêm bản MP4 để so sánh	65,2 MB	Cần đăng nhập

Mã	Tài nguyên	Định dạng	Dung lượng	Quyền
R2	Bảng phiên âm và 300 từ HSK 1	PDF 6 trang	410 KB	Công khai
R3	Gói nghe bài 1 "Chào hỏi", 10 từ	MP3 128 kbit/s, 27 giây	426 KB	Công khai

Trang có đủ 4 trạng thái: đang tải, có dữ liệu, trống, lỗi có nút thử lại. Video phát bằng thư viện hls.js; liên kết xin ngay lúc bấm vì có hạn 10 phút. Học liệu dựng lại được bằng lệnh make sample; bitrate ép đúng bảng tính để phép đo băng thông có nghĩa.

4.5 Giám sát và cảnh báo

Prometheus thu số liệu của API (thời gian xử lý từng loại request, số kết nối CSDL đang chờ), PostgreSQL, Redis và một bộ dò gọi thử từ bên ngoài. Grafana có 3 bảng: chất lượng dịch vụ, mở rộng, dữ liệu. Nhật ký request vào Loki. Alertmanager báo khi API ngừng trả lời, khi gọi từ ngoài không được, khi bản sao CSDL trễ hoặc khi bản sao lưu quá cũ.

4.6 Sao lưu và xử lý sự cố

Mỗi ngày lúc 02:00: xuất CSDL bằng pg_dump, nén học liệu, tự kiểm tra bản vừa tạo, ghi mã băm, giữ 14 bản. Nhật ký thay đổi của CSDL (WAL) được lưu liên tục, tối đa 60 giây một lần, để khôi phục về đúng thời điểm. Script chép bản mới nhất ra ngoài cụm và so lại mã băm.

Sự cố	Phát hiện bằng	Cách xử lý	Mục tiêu
API ngừng trả lời	Cảnh báo gọi từ ngoài không được	Kiểm tra pod, bật lại hoặc tăng số bản	Có trả lời lại trong vài giây sau khi bật
Mất dữ liệu CSDL	Lỗi khi đọc ghi, số dòng bất thường	Khôi phục bản sao lưu mới nhất vào CSDL sạch, so số dòng	Mất tối đa 1 ngày (bản dump) hoặc 1 phút (có WAL)
Mất 1 tệp học liệu	Kiểm thử mã băm báo sai	Lấy lại từ gói học liệu đã sao lưu, so mã băm	Học liệu đúng 100%
Đĩa sao lưu đầy	Job sao lưu báo thất bại	Dọn bản cũ hoặc thêm đĩa, chạy lại job	Không để bản lỗi đè bản tốt

Kiểm thử và đánh giá

5 Kiểm thử

5.1 Cách đo

Mục	Quy định
Công cụ tạo tải	k6 0.57 trong Docker, giữ đều số lượt mới mỗi giây: máy chủ chậm thì k6 vẫn gửi đúng nhịp, nên thấy được hàng chờ thật
Tiêu chí đạt	95% lượt đọc dưới 300 ms (gọi là p95), 95% lượt ghi dưới 500 ms, lỗi dưới 0,5%, không bỏ lượt nào

Mục	Quy định
Lặp lại	Mỗi lần: chờ API ổn định, làm nóng 1 phút, đo 5 phút; mỗi mức 3 lần; tự ghi điều kiện đo, CPU/RAM mỗi 5 giây
Loại lần đo hỏng	Lần nào máy hết RAM phải dùng swap thì bỏ, đo lại
Máy đo	GitHub Actions Ubuntu 24.04.5 LTS, AMD EPYC 9V74 80-Core Processor, 4 luồng, 15983 MB RAM; máy laptop của nhóm không đủ RAM nên đo chính thức trên GitHub Actions; lab dựng lại theo README mỗi lượt; k6 chạy chung máy với hệ thống

5.2 Kiểm thử 1: máy khách tải đúng học liệu

Ca	Thao tác	Kết quả
Trình duyệt	Khách mở trang, tải PDF; đăng nhập; phát video; nghe bài	Đạt (07/11): video phát ở 540p, bài nghe 27 giây, 0 lỗi
Mã băm	scripts/kt1.sh tải đủ 30 tệp (gồm 22 đoạn video), so với manifest.sha256	Đạt 3 lần: 30 tệp đúng, 0 tệp sai, 69174525 byte, 1.7 giây (09/11)
Test tự động	5 ca: có dữ liệu, trống, lỗi rồi thử lại, khách được mời đăng nhập, không xin được liên kết	5/5 đạt
Giao diện	Màn 360, 390, 1440 px; sáng và tối	Không tràn ngang, nút bấm đủ lớn, đi được bằng bàn phím
Lighthouse	3 lần mỗi chế độ	Điện thoại: hiệu năng 87, 89, 89, truy cập 100; máy tính: 100 cả 4 nhóm

5.3 Kiểm thử 2: ba mức tải

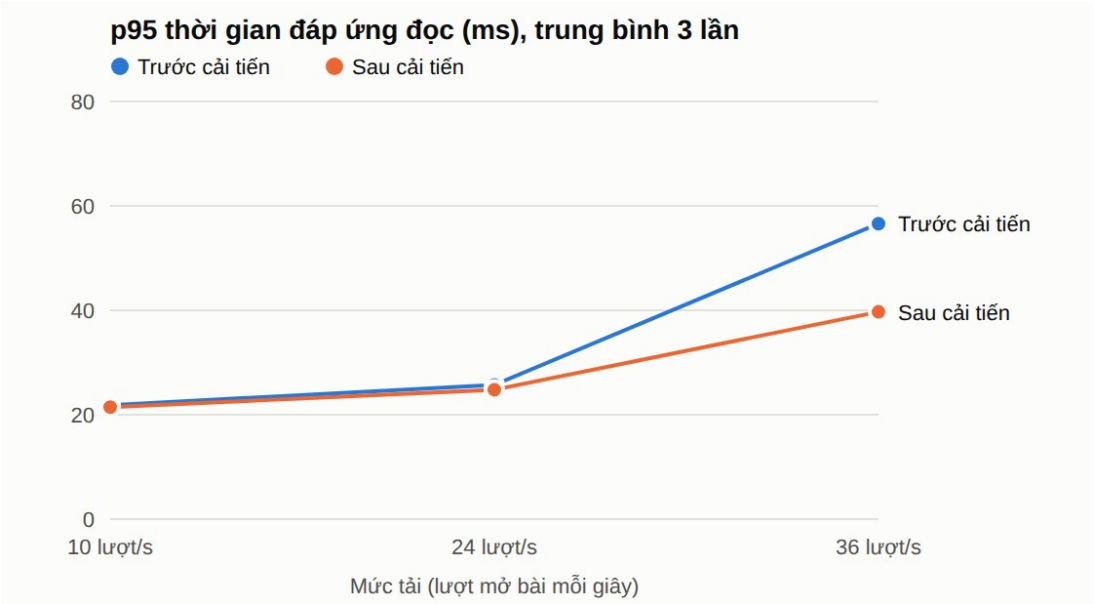
Đo thăm dò để chọn mức: tăng số lượt mở bài mỗi giây (mỗi lượt 5 request đọc), mỗi mức 1 phút.

Lượt/giây	Request/giây	p95 (ms)	Lỗi	Đạt
10	49,5	23,2	0,00%	Có
20	98,9	23,9	0,00%	Có
30	148,1	30,9	0,00%	Có
40	197,0	52,1	0,00%	Có
50	244,7	261,4	1,09%	Không
60	292,5	688,0	24,85%	Không
70	341,8	767,8	37,79%	Không
80	386,5	824,7	46,53%	Không

Lỗi từ 50 lượt/giây trở lên đều là mã 503 "thử lại sau" do hệ thống chủ động từ chối khi quá tải, không có lỗi máy chủ nào khác. Mức cao nhất còn đạt là 40, nên 3 mức của Kiểm thử 2 lấy 25%, 60%, 90% mức

đó: **10, 24, 36 lượt/giây**, khoảng 50, 120, 180 request/giây. Bản "trước" là bản chưa sửa điểm nghẽn (build từ commit 2693172), đo cùng máy, cùng lượt chạy với bản "sau".

Mức	Bản	Request/giây	p50 (ms)	p95 (ms) ± độ lệch 3 lần	p99 (ms)	Lỗi	CPU API (lõi)
10/giây	Trước	49,9	7	22 ± 0	28	0,00%	0,49
10/giây	Sau	49,9	6	21 ± 0	28	0,00%	0,46
24/giây	Trước	119,6	7	26 ± 1	42	0,00%	1,11
24/giây	Sau	119,7	7	25 ± 1	40	0,00%	1,06
36/giây	Trước	179,5	12	57 ± 16	104	0,00%	1,81
36/giây	Sau	179,5	9	40 ± 2	75	0,06%	1,65



Hình 3. p95 thời gian đọc theo mức tải, trước và sau khi sửa điểm nghẽn; ngưỡng đạt 300 ms nằm xa phía trên trục

Cả 3 mức đều đạt ở cả 2 bản. Ở 2 mức thấp, 2 bản gần như bằng nhau vì chưa chạm giới hạn nào. Ở mức cao, bản sau nhanh hơn (57 → 40 ms) và ổn định hơn nhiều (độ lệch 16 → 2 ms). Số gốc: data/raw/gha-09-11/ket-qua-kt2/; tổng hợp: data/kt2-tong-hop.xlsx.

5.4 Kiểm thử 3: dừng dịch vụ và khôi phục

Lần	Dừng hẳn API lúc (UTC)	Cảnh báo từ ngoài	Bật lại tới khi có trả lời	Học liệu sau khi bật
1	2026-11-09 11:48:36	63 giây	5 giây	Đúng 30/30 tệp
2	2026-11-09 11:51:43	84 giây	5 giây	Đúng 30/30 tệp
3	2026-11-09 11:55:00	84 giây	4 giây	Đúng 30/30 tệp

Thời gian gián đoạn thực tế do người vận hành quyết định (dừng bao lâu thì gián đoạn bấy lâu). Phần đo được là thời gian **phát hiện** (cảnh báo kêu sau khoảng 1 phút) và thời gian **khôi phục** (bật lại tới khi có trả lời 4–5 giây).

Ca	Cách làm	Kết quả
Mất dữ liệu	Sinh 1 triệu dòng học và 600 bài thi giả, sao lưu, ghi thêm 1 dòng, xoá hẳn CSDL và 1 tệp học liệu, khôi phục	Số dòng 6 bảng chính khớp; dòng ghi sau lúc sao lưu bị mất, đúng như RPO của bản dump; RTO 52 giây từ lúc xoá tới khi kiểm tra xong
Tắt từng thành phần	scripts/chaos.sh, dò mỗi giây	tắt 1 bản API 0 s, tắt Redis 0 s, tắt Postgres 16 s, tắt 1 node 94 s
Mất CSDL chính khi đang ghi 30 lượt/giây	Tắt đột ngột CSDL chính, chuyển sang bản sao	4.416 lượt đã xác nhận, mất 0; chuyển xong sau 3 s; kiểm tra sẵn sàng lỗi tối đa 3 s
Xoá nhầm, khôi phục về đúng thời điểm trước đó	Bản sao vật lý + nhật ký thay đổi (WAL)	lấy lại 1.000 dòng trong 6 giây
Sao lưu gặp lỗi	Đích hết chỗ, đích chỉ đọc, không tới được CSDL	3/3 trường hợp báo lỗi đúng, bản cũ không bị ảnh hưởng

5.5 Kiểm thử phân quyền

Chạy bằng tài khoản giả, không dùng quyền quản trị (scripts/kiem-quyen.sh, 09/11). 11/11 dòng đạt:

Ca	Mong đợi	Thực tế
Khách xin liên kết video cần đăng nhập	401	401
Dùng liên kết đã hết hạn	410	410
Sửa chữ ký trên liên kết	403	403
Người học xin liên kết video đúng quyền	200	200
Người học xem tiến độ lớp của người khác	403 hoặc 404	403
Máy ngoài cụm nối thẳng PostgreSQL, Redis	Đóng	Đóng
API nối PgBouncer / nối thẳng PostgreSQL	Được / bị chặn	Được / bị chặn
Từ ngoài gọi trang số liệu nội bộ /api/metrics	404	404
Người học gọi chức năng của quản trị	403	403

5.6 Đo thêm

Phép đo	Điều kiện	Kết quả
Đăng nhập đồng loạt	Mỗi lượt 1 tài khoản khác, mỗi mức 3 lần × 1 phút	10 lượt/giây: p95 55 ms, 0 lỗi; 20 lượt/giây: p95 58 ms, 0 lỗi; 40 lượt/giây: p95 72 ms, 0 lỗi. 1.000 sinh viên đăng nhập trong 1 phút chỉ cần khoảng 17 lượt/giây
Nộp bài thi thử đồng loạt	600 người mở đề trong 60 giây rồi nộp trong 30 giây, 10% gửi lại 2 lần	601, 600, 601 bài mới ở 3 lần, đúng bằng số lượt gửi; gửi lại không tạo bài thứ 2
Tự thêm bản API	Tăng dần tới 200 request/giây	4 bản → 3 → 5 → 6 bản; Kubernetes thêm bản khi CPU vượt 65%
Phát video	Tải đoạn video 540p từ Nginx, 2 phút mỗi mức	60 người xem: 126 Mbps; 150 người xem: 315 Mbps; 300 người xem: 631 Mbps; API không tham gia. Đo trong 1 máy nên chỉ chứng minh Nginx đẩy được, chưa chứng minh mạng thật

5.7 Mở rộng tự chọn: thay 1 thành phần, so cùng tiêu chí

Cùng video 66 giây	Hình đầu tiên	Tải trong 60 giây	Dừng chờ tải	Xem được	Chất lượng
Mạng thường, HLS	362 ms	38,2 MB	0 lần, 0,0 giây	59,7 giây	540 rồi 720p
Mạng thường, MP4	95 ms	37,6 MB	0 lần, 0,0 giây	59,9 giây	720p
Mạng chậm, HLS	8.267 ms	11,7 MB	6 lần, 12,5 giây	39,3 giây	540p
Mạng chậm, MP4	843 ms	12,0 MB	8 lần, 28,1 giây	31,1 giây	720p

MP4 lên hình nhanh hơn vì không phải tải danh sách đoạn và thư viện phát. Trên mạng chậm, HLS tự hạ xuống 540p nên ít phải dừng chờ và xem được nhiều hơn. Kết luận theo số đo: dùng HLS cho bài giảng, MP4 chỉ hợp đoạn ngắn trên mạng tốt.

Cách sao lưu	Thời gian sao lưu	Dung lượng	Thời gian khôi phục	Khôi phục 1 bảng	Về đúng thời điểm
pg_dump (xuất dữ liệu)	12,0 giây	82 MB	32,7 giây	Được	Không
pg_basebackup (chép cả ổ)	5,0 giây	556 MB	6,9 giây	Không	Được, kèm WAL

Trung bình 3 lần trên CSDL có 1 triệu dòng; số dòng sau khôi phục khớp cả 2 cách. Nhóm dùng cả 2: bản xuất hằng ngày để chép ra ngoài (nhỏ, khôi phục từng bảng được), bản chép cả ổ kèm nhật ký thay đổi để chỉ mất tối đa khoảng 1 phút dữ liệu.

Cách tạo tải, 3 lần mỗi mức	Mở: request/giây	Mở: p95	Đóng: request/giây	Đóng: p95
15 lượt/giây	74,7	27 ms	79,4	38 ms
30 lượt/giây	149,3	40 ms	146,3	61 ms

Cách "mở" gửi đều số lượt mới mỗi giây; cách "đóng" có số người ảo cố định, mỗi người chờ trả lời xong mới gửi tiếp. Kết quả ngược với dự đoán ban đầu: khi chưa quá tải, cách đóng còn cho p95 cao hơn vì người ảo gửi dồn ngay khi lượt trước xong. Cách đóng chỉ che độ chậm khi máy chủ đã quá tải (nó tự gửi chậm lại), nên nhóm dùng cách mở cho mọi phép đo quá tải.

6 Đánh giá

6.1 Điểm nghẽn và cải tiến

Tăng tải dần trên 6 bản API (kịch bản đọc: mở lộ trình học, tra từ) tới 400 request/giây rồi giữ 60 giây, cùng máy với Kiểm thử 2:

Bản	Mức cao nhất còn đạt	Giữ 400 request/giây	Điểm nghẽn
Trước khi sửa	157 request/giây	Phục vụ được 177/giây nhưng 95% lượt chờ tới **5,0 giây**; 430 request xếp hàng chờ kết nối CSDL	Hàng chờ kết nối CSDL; trang lộ trình học đọc lại cùng một nội dung mỗi lượt
Sau khi sửa	190 request/giây	Phục vụ được 167/giây, 95% lượt chờ **0,73 giây**; phần vượt nhận "thử lại sau"	CPU của máy 4 nhân; chỉ cần thêm bản API hoặc thêm máy

Cải tiến gồm 3 phần dễ hiểu: (1) lưu tạm 60 giây khung lộ trình học vì ai xem cũng giống nhau; (2) khi hàng chờ CSDL dài, trả lời ngay "thử lại sau" cho lượt đọc; (3) kiểm tra sức khoẻ chờ lâu hơn để Kubernetes không khởi động lại nhầm bản đang bận. Kết quả: mức tối đa còn nhanh tăng khoảng 21%, và quá tải không còn làm mọi người cùng chờ 5 giây. Đánh đổi: một phần lượt đọc bị từ chối thay vì chờ.

6.2 Vì sao thử 20 kết nối không chứng minh phục vụ được 1.000 sinh viên

- **Thời gian trả lời không tăng đều theo tải.** Từ 10 tới 40 lượt/giây, p95 chỉ tăng từ 23 lên 52 ms; thêm 10 lượt nữa đã lên 261 ms. Mức nhỏ chưa chạm giới hạn nên không nói gì về điểm gãy.
- **Điểm nghẽn chỉ lộ ra khi tải đủ lớn:** hàng chờ kết nối CSDL, rồi CPU của API, rồi lỗi khởi động lại nhầm bản đang bận đều không xuất hiện ở vài chục kết nối.
- **Tài khoản khác người dùng cùng lúc, người dùng khác request:** 1.000 tài khoản sinh ra 63, 300 hoặc 600 người cùng lúc tuỳ tình huống, nặng nhất là các đợt đồng loạt.
- **Kết quả chỉ đúng cho đúng điều kiện đo** (cùng máy, cùng dữ liệu, cùng cấu hình); muốn suy rộng phải nêu điều kiện và giới hạn như mục 6.3.

6.3 Giới hạn

- Lab chạy trên 1 máy ảo 4 nhân, k6 tranh CPU với hệ thống, nên điểm gãy thật có thể cao hơn. Mỗi lượt máy ảo GitHub có thể khác loại CPU, nên chỉ so trước và sau trong cùng 1 lượt chạy.
- Băng thông đo trong 1 máy chỉ chứng minh Nginx đẩy được bao nhiêu, chưa chứng minh mạng thật chịu được khi cả lớp xem video.
- Dữ liệu giả nhỏ hơn dữ liệu thật sau 1 năm. Bảng nhật ký học đã chia theo tháng: truy vấn trên 1 triệu dòng từ 280,6 ms còn 7,4 ms.
- Chuyển sang CSDL bản sao hiện làm bằng tay; thời gian phát hiện sự cố chưa tính vào 3 giây chuyển đổi.

6.4 Mở rộng theo quy mô

Chỉ tiêu	100 tài khoản	1.000	10.000	100.000
Tự học cùng lúc	7	63	625	6.250
Xem video cùng giờ	60	300	1.500	5.000
Thi thử đồng loạt	80	600	3.000	10.000
Băng thông video, có dự phòng (Mbps)	178	888	4.441	14.802
Số bản API ước lượng	2	2	5	50

Quy mô	Việc phải làm	Chưa cần	Chuyển mức khi
100	Giới hạn request theo tài khoản; sao lưu đã thử khôi phục	Kubernetes, bản sao, CDN	Máy hết RAM liên tục hoặc p95 đọc vượt 300 ms
1.000	Nhiều bản API tự thêm theo CPU, worker riêng, PgBouncer, video qua Nginx/CDN, giám sát đầy đủ	Chia nhỏ CSDL	Đã đủ 6 bản mà CPU vẫn trên 70%
10.000	CSDL máy riêng + bản sao, kho tệp + CDN bắt buộc, hàng đợi việc, chia bảng lớn theo tháng	Chia dữ liệu theo người dùng	CPU CSDL chính trên 70% lúc cao điểm
100.000	Nhiều vùng, tự chuyển CSDL khi hỏng bằng công cụ có sẵn, tách kho phân tích	Tự viết cơ chế đồng thuận	Chi phí và số người vận hành

Phân công và tài liệu tham khảo

7 Phân công và cách dùng AI

Thành viên	Vai trò	Sản phẩm kỹ thuật chính
Phạm Hải Đăng	Trưởng nhóm kỹ thuật, Solution Architect, Platform/DevOps, Backend	Bảng tính tài nguyên, kiến trúc, lab k3s, API học liệu, giám sát, cải tiến theo điểm nghẽn
Lê Đình Thành	Frontend, UI/UX, Media/CDN, kiểm thử giao diện và đo trình duyệt	3 học liệu, trang /hoc-lieu, Nginx phát học liệu, HAR, Lighthouse, so sánh HLS và MP4
Bùi Thị Vân Anh	QA Lead, kiểm thử hiệu năng, phân tích số liệu	Kịch bản k6, Kiểm thử 1 và 2, tổng hợp số liệu, so sánh 2 cách tạo tải
Chu Hồng Đức	SRE: sao lưu, khôi phục, diễn tập sự cố; kiểm thử phân quyền	Script sao lưu, Kiểm thử 3, kiểm thử phân quyền, đánh giá mở rộng, README

Cách dùng AI: nhóm dùng trợ lý lập trình Claude Code (Anthropic) để soạn bản nháp mã hạ tầng, kịch bản đo, script diễn tập, chạy đo trên lab và soạn bản nháp tài liệu. Mọi số liệu lấy từ tệp gốc trong data/raw/ và evidence/, không có số nào do AI tự đặt ra. Mỗi thành viên đọc hiểu, chạy lại phần của mình và chịu trách nhiệm giải thích khi bảo vệ.

Thành phần	Nguồn tham khảo	Nhóm làm gì với nó
Cụm k3s trong Docker	Tệp compose mẫu trong tài liệu k3s	Thêm IP cố định, giới hạn RAM, giữ mật khẩu node khi tạo lại
Các dịch vụ	Image chính thức của PostgreSQL, Redis, Nginx, PgBouncer, Prometheus, Grafana, Loki, Jaeger	Viết cấu hình, giới hạn tài nguyên, chặn mạng, luật cảnh báo
Tạo tải	k6 của Grafana Labs	Viết 8 kịch bản, script chạy lặp và thu CPU/RAM
Phát video	Thư viện hls.js	Gắn vào trang học liệu
Mã ứng dụng	Hanni, do Đăng viết từ trước	Thêm số liệu đo, lưu tạm, chống gửi trùng bài, từ chối khi quá tải

8 Tài liệu tham khảo

1. M. Kleppmann. Designing Data-Intensive Applications. O'Reilly, 2017.

2. B. Beyer và cộng sự. Site Reliability Engineering. O'Reilly, 2016 (phần mục tiêu chất lượng dịch vụ và cảnh báo).

3. J. D. C. Little. "A Proof for the Queuing Formula: L = λW". Operations Research, 1961.

4. Apple. HTTP Live Streaming (HLS) Authoring Specification for Apple Devices.

5. Tài liệu Kubernetes: Horizontal Pod Autoscaling; Liveness, Readiness and Startup Probes; Network Policies.

6. Tài liệu k3s: Running K3s in Docker.

7. Tài liệu Grafana k6: Open and closed models; Scenarios.

8. Tài liệu PostgreSQL 16: pg_dump; pg_basebackup; Continuous Archiving and Point-in-Time Recovery; Streaming Replication.

9. Tài liệu PgBouncer: Pool modes. Tài liệu Nginx: ngx_http_secure_link_module.

10. Dữ liệu: krmanik/HSK-3.0 (CC BY-SA 4.0), CVDICT (CC BY-SA 4.0), Unicode Unihan Database (Unicode License V3).

Phụ lục cấu hình

Phụ lục không tính vào giới hạn 12–18 trang của báo cáo.

Phụ lục A. Dựng lại và chạy kiểm thử

Lệnh (chạy trong cau-hinh-ma-nguon/hanni-server/infra)	Việc
`make up && make images && make deploy && make seed && make observability`	Dựng lab từ đầu
`scripts/kt1.sh k6/ket-qua/kt1.txt`	Kiểm thử 1
`scripts/chay-kt2.sh k6/ket-qua/kt2 "10 24 36" 3`	Kiểm thử 2
`scripts/kt3-dung-dich-vu.sh k6/ket-qua/kt3 3`	Kiểm thử 3, dừng dịch vụ
`scripts/kt3-mat-du-lieu.sh`	Kiểm thử 3, mất dữ liệu
`scripts/kiem-quyen.sh k6/ket-qua/quyen.txt`	Kiểm thử phân quyền

Phụ lục B. Cấu hình chính

Sửa ở tệp (trong cau-hinh-ma-nguon/hanni-server/infra), chạy make deploy; tệp config.yaml cần thêm lệnh khởi động lại API. Bảng đầy đủ: infra/README.md, mục "Sửa nhanh khi bảo vệ"; phiên bản và cách chạy web: README.md của hồ sơ.

Thông số	Giá trị trong lab	Ý nghĩa	Sửa ở tệp
minReplicas, maxReplicas, averageUtilization	2, 6, 65%	Kubernetes tự thêm hoặc bớt bản API theo CPU	k8s/app/api.yaml
resources (requests, limits)	0,25–1 lõi, 256–512 MB	CPU, RAM cho mỗi bản API	k8s/app/api.yaml
THROTTLE_LIMIT, THROTTLE_TTL	300 request / 60 giây	Giới hạn request theo tài khoản	k8s/app/ config.yaml
SHED_POOL_WAITING, SHED_LOOP_LAG_MS	8 request chờ, 250 ms	Quá tải thì trả lời "thử lại sau" cho lượt đọc (0 là tắt)	k8s/app/ config.yaml
RESOURCE_LINK_TTL_SEC	600	Liên kết học liệu hết hạn sau 10 phút	k8s/app/ config.yaml
Danh sách học liệu, requiresLogin	3 học liệu	Học liệu hiện trên trang, cần đăng nhập hay không	k8s/media-manifest.json
schedule, BACKUP_KEEP	0 19 * * * (02:00 giờ VN), 14 bản	Giờ sao lưu hằng ngày, số bản giữ lại	k8s/backup/ cronjob.yaml
Kiểm tra sức khoẻ (liveness)	Chờ 5 giây, lỗi 6 lần mới khởi động lại	Không khởi động lại nhầm bản đang bận	k8s/app/api.yaml
Ngưỡng cảnh báo	p95 đọc 300 ms, dò từ ngoài 30 giây	Khi nào Alertmanager báo	observability/ rules/hanni-slo.yml